SQL & PL/SQL

Guide Complet avec Exercices

M1 SSI – USTHB – Module ASGBD – Dr. Ferrahi

Ce guide couvre :

- Types de donnÃľes Oracle
- DDL : CREATE / ALTER / DROP
- DML : INSERT / UPDATE / DELETE
- DQL : SELECT complet
- Contraintes d'intÃľgritÃľ
- Vues et sÃľquences
- Index (B-tree, Bitmap, ...)
- Tablespaces
- Utilisateurs & privilÃĺges
- RÃťles
- Profils
- PL/SQL : blocs, variables
- Structures de contrÃťle
- Curseurs
- ProcÃľdures et fonctions
- Triggers
- Exceptions
- Dictionnaire de donnÃľes

Structure : pour chaque concept → explication → syntaxe dÃľtaillÃľe → exemple → exercice(s). Toutes les corrections sont regroupÃľes en fin de document.

2025/2026

Table des matières

1 Types de donnÃľes Oracle **3**

2 DDL – Langage de DÃľfinition de DonnÃľes **3**
2.1 CREATE TABLE . . . 3
2.2 ALTER TABLE . . . 5
2.3 DROP TABLE et TRUNCATE . . . 6

3 DML – Langage de Manipulation de DonnÃľes **6**
3.1 INSERT . . . 6
3.2 UPDATE . . . 7
3.3 DELETE . . . 8

4 DQL – SELECT : Interrogation complÃÍte **9**
4.1 Syntaxe gÃľnÃľrale . . . 9
4.2 Fonctions d'agrÃľgation . . . 9
4.3 Fonctions SQL utiles . . . 10

5 Vues et SÃľquences **11**
5.1 Vues . . . 11
5.2 SÃľquences . . . 12

6 Index **12**
6.1 Index B-arbre (par dÃľfaut) . . . 13
6.2 Index Bitmap . . . 13
6.3 Index sur clÃľ ÃľtrangÃÍre (bonne pratique) . . . 13

7 Tablespaces **14**

8 Utilisateurs et PrivilÃÍges **15**
8.1 Gestion des utilisateurs . . . 15
8.2 PrivilÃÍges systÃÍme . . . 16
8.3 PrivilÃÍges objet . . . 16

9 RÃťles **18**

10 Profils **19**

11 PL/SQL – Introduction et Blocs **20**
11.1 Structure d'un bloc PL/SQL . . . 20

11.2 DÃľclaration de variables 21
11.3 SELECT INTO (mono-ligne) 21

12 Structures de ContrÃťle PL/SQL **22**
12.1 IF / ELSIF / ELSE 22
12.2 CASE 23
12.3 Boucles 23

13 Curseurs PL/SQL **24**
13.1 Curseur explicite 24
13.2 Curseur avec paramÃĺtre 25
13.3 Boucle FOR sur curseur (syntaxe simplifiÃľe) 25

14 ProcÃľdures et Fonctions StockÃľes **26**
14.1 ProcÃľdures 26
14.2 Fonctions 27

15 Triggers (DÃľclencheurs) **29**

16 Gestion des Exceptions PL/SQL **32**

17 Dictionnaire de DonnÃľes Oracle **33**

1 Types de donnÃľes Oracle

Oracle dispose d'un ensemble riche de types. Voici ceux utilisÃľs dans le cours :

Type	Utilisation	Exemple
`NUMBER(p,s)`	Nombre : p chiffres totaux, s dÃľcimales	`NUMBER(8,2)`
`NUMBER(p)`	Entier Ãă p chiffres	`NUMBER(4)`
`VARCHAR2(n)`	ChaÃőne variable, max n caractÃĺres	`VARCHAR2(50)`
`CHAR(n)`	ChaÃőne fixe, exactement n caractÃĺres	`CHAR(1)`
`DATE`	Date + heure (JJ/MM/AAAA HH :MM :SS)	`DATE`
`BOOLEAN`	Vrai/Faux (PL/SQL seulement)	`BOOLEAN`
`CLOB`	Texte long (> 4000 chars)	`CLOB`
`BLOB`	DonnÃľes binaires	`BLOB`

Note importante

`VARCHAR2` est prÃľfÃľrÃľ Ãă `CHAR` en pratique car il n'occupe que l'espace rÃľellement utilisÃľ. `CHAR(5)` pour la valeur `'Ali'` stocke `'Ali  '` (avec 2 espaces), ce qui peut causer des surprises dans les comparaisons.

2 DDL – Langage de DÃľfinition de DonnÃľes

2.1 CREATE TABLE

Syntaxe CREATE TABLE

```
CREATE TABLE [schema.]nom_table (
  col1   type1  [DEFAULT valeur]  [NOT NULL],
  col2   type2  [DEFAULT valeur]  [NOT NULL],
  ...
  [CONSTRAINT nom_pk  PRIMARY KEY (col1 [, col2])]
  [CONSTRAINT nom_uq  UNIQUE      (col1 [, col2])]
  [CONSTRAINT nom_fk  FOREIGN KEY (col)
                      REFERENCES table_parent(col_parent)
                      [ON DELETE {CASCADE | SET NULL}]]
  [CONSTRAINT nom_chk CHECK (condition)]
) [TABLESPACE nom_ts];
```

DÃľtail de chaque contrainte :

- `PRIMARY KEY` : identifie uniquement chaque ligne. Implique NOT NULL + UNIQUE. Oracle crÃľe automatiquement un index unique dessus.
- `UNIQUE` : toutes les valeurs de la colonne doivent Ãłtre diffÃľrentes. NULL est autorisÃľ (plusieurs NULL possibles).
- `NOT NULL` : la colonne ne peut pas contenir NULL. Se dÃľclare au niveau colonne seulement.

- `FOREIGN KEY ... REFERENCES` : valeur doit exister dans la table parente.
  - `ON DELETE CASCADE` : si le parent est supprimÃľ, les enfants sont supprimÃľs automatiquement.
  - `ON DELETE SET NULL` : si le parent est supprimÃľ, la FK de l'enfant passe Ãă NULL.
  - (sans option) : la suppression du parent est refusÃľe si des enfants existent.
- `CHECK(condition)` : impose une rÃĺgle logique. Exemples : `CHECK(age BETWEEN 18 AND 60)`, `CHECK(grade IN ('A','B','C'))`.

Exemple complet :

```sql
CREATE TABLE Employe (
  NSS           NUMBER(8)      CONSTRAINT pk_emp PRIMARY KEY,
  Nom           VARCHAR2(40)   NOT NULL,
  Prenom        VARCHAR2(40),
  DateRecruit   DATE           DEFAULT SYSDATE,
  Salaire       NUMBER(10,2)   DEFAULT 0,
  Categorie     VARCHAR2(20),
  Age           NUMBER(3),
  NumDept       NUMBER(4),
  CONSTRAINT uq_emp_nom    UNIQUE (Nom, Prenom),
  CONSTRAINT chk_cat       CHECK (Categorie IN ('Mecanicien','Assistant')),
  CONSTRAINT chk_age       CHECK (Age BETWEEN 18 AND 65),
  CONSTRAINT fk_emp_dept   FOREIGN KEY (NumDept)
                           REFERENCES Departement(NumDept)
                           ON DELETE SET NULL
);
```

Exercice 1 – CREATE TABLE

Soit le schÃľma suivant pour une gestion d'une bibliothÃĺque :

- **AUTEUR**(NumAut, NomAut, PrenomAut, Nationalite)
- **LIVRE**(ISBN, Titre, NumAut*, Annee, Prix, Genre)
- **ABONNE**(NumAbo, NomAbo, Email, DateInscription, Statut)
- **EMPRUNT**(NumAbo*, ISBN*, DateEmprunt, DateRetour, DateRetourEff)

Contraintes supplÃľmentaires :

- Le `Genre` d'un livre est parmi : 'Roman', 'Science', 'Histoire', 'Informatique'.
- Le `Prix` doit Ãłtre supÃľrieur Ãă 0.
- Le `Statut` d'un abonnÃľ est 'Actif' ou 'Suspendu'.
- La `DateRetour` doit Ãłtre supÃľrieure Ãă la `DateEmprunt`.
- Si un auteur est supprimÃľ, ses livres doivent Ãłtre supprimÃľs en cascade.
- Si un abonnÃľ est supprimÃľ, ses emprunts passent Ãă NULL (uniquement NumAbo).

ÃĽcrire les 4 scripts CREATE TABLE complets.

2.2 ALTER TABLE

Syntaxe ALTER TABLE

```sql
-- Ajouter une colonne
ALTER TABLE nom_table ADD nom_col type [DEFAULT val] [NOT NULL];

-- Modifier le type/taille d'une colonne (possible si vide ou compatible)
ALTER TABLE nom_table MODIFY nom_col nouveau_type;

-- Renommer une colonne
ALTER TABLE nom_table RENAME COLUMN ancien_nom TO nouveau_nom;

-- Supprimer une colonne
ALTER TABLE nom_table DROP COLUMN nom_col;

-- Ajouter une contrainte
ALTER TABLE nom_table ADD CONSTRAINT nom_c type_c (col) [...];

-- DÃľsactiver une contrainte (les donnÃľes existantes ne sont plus vÃľrifiÃľes)
ALTER TABLE nom_table DISABLE CONSTRAINT nom_c;

-- RÃľactiver une contrainte
ALTER TABLE nom_table ENABLE CONSTRAINT nom_c;

-- Supprimer une contrainte
ALTER TABLE nom_table DROP CONSTRAINT nom_c;

-- Renommer la table
ALTER TABLE ancien_nom RENAME TO nouveau_nom;
```

Listing 1 – Exemples ALTER TABLE

```sql
-- Ajouter une colonne DateNaissance
ALTER TABLE Employe ADD DateNaissance DATE;

-- Agrandir le champ Nom
ALTER TABLE Employe MODIFY Nom VARCHAR2(80);

-- Ajouter une contrainte CHECK aprÃľs coup
ALTER TABLE Employe ADD CONSTRAINT chk_salpos CHECK (Salaire >= 0);

-- DÃľsactiver pour permettre une mise Ãă jour spÃľciale
ALTER TABLE Interventions DISABLE CONSTRAINT chk_dates;
UPDATE Interventions SET DateDebut = DateDebut + 5 WHERE ...;
ALTER TABLE Interventions ENABLE CONSTRAINT chk_dates;

-- Supprimer la colonne DateNaissance
ALTER TABLE Employe DROP COLUMN DateNaissance;
```

Attention

MODIFY ne peut pas rÃľduire la taille d'une colonne si des donnÃľes existent et risquent d'Ãłtre tronquÃľes. On ne peut pas passer une colonne en NOT NULL si elle contient dÃľjÃă des NULL.

Exercice 2 – ALTER TABLE

En partant de la table `LIVRE` crÃľÃľe Ãă l'exercice 1 :

1. Ajouter une colonne `NbPages` de type entier, valeur par dÃľfaut 0.
2. Renommer la colonne `Annee` en `AnneePublication`.
3. Modifier la taille de `Titre` pour qu'elle accepte 200 caractÃľres.
4. Ajouter une contrainte qui impose que `NbPages > 0`.
5. DÃľsactiver cette contrainte, insÃľrer un livre avec NbPages = 0, puis rÃľactiver la contrainte. Qu'est-ce qui se passe ?

2.3 DROP TABLE et TRUNCATE

```
-- Supprimer une table et toutes ses donnÃľes (irrÃľversible)
DROP TABLE Employe;

-- Avec CASCADE CONSTRAINTS : supprime aussi les FK qui la rÃľfÃľrencent
DROP TABLE Employe CASCADE CONSTRAINTS;

-- TRUNCATE : vider la table sans supprimer sa structure (plus rapide que
   DELETE)
TRUNCATE TABLE Employe;
-- TRUNCATE ne peut pas Ãłtre annulÃľ (pas de ROLLBACK possible)
```

3 DML – Langage de Manipulation de DonnÃľes

3.1 INSERT

Syntaxe INSERT

```
-- InsÃľrer en prÃľcisant les colonnes (recommandÃľ)
INSERT INTO nom_table (col1, col2, col3) VALUES (val1, val2, val3);

-- InsÃľrer dans toutes les colonnes (dans l'ordre de la table)
INSERT INTO nom_table VALUES (val1, val2, val3, ...);

-- InsÃľrer depuis une autre table
INSERT INTO nom_table (col1, col2)
  SELECT col_a, col_b FROM autre_table WHERE condition;

-- InsÃľrer plusieurs lignes (Oracle 23c+, sinon utiliser INSERT ALL)
INSERT ALL
```

```
  INTO Employe (NSS, Nom, Salaire) VALUES (101, 'Ahmed',  30000)
  INTO Employe (NSS, Nom, Salaire) VALUES (102, 'Fatima', 28000)
  INTO Employe (NSS, Nom, Salaire) VALUES (103, 'Karim',  35000)
SELECT * FROM DUAL;
```

Listing 2 – Exemples INSERT

```
-- Insertion simple
INSERT INTO Auteur (NumAut, NomAut, Nationalite)
VALUES (1, 'Hugo', 'FranÃğaise');

-- NULL explicite pour une colonne optionnelle
INSERT INTO Employe (NSS, Nom, Prenom, Salaire)
VALUES (1001, 'Benali', 'Omar', NULL);

-- Utiliser SYSDATE
INSERT INTO Emprunt (NumAbo, ISBN, DateEmprunt, DateRetour)
VALUES (5, '978-2-07-040850-4', SYSDATE, SYSDATE + 14);

-- Utiliser TO_DATE pour un format prÃľcis
INSERT INTO Employe (NSS, Nom, DateRecruit)
VALUES (1002, 'Amara', TO_DATE('15/03/2023', 'DD/MM/YYYY'));
```

Exercice 3 – INSERT

Avec la BD bibliothÃĺque :

1. InsÃľrer 3 auteurs de nationalitÃľs diffÃľrentes.
2. InsÃľrer 4 livres (dont au moins 2 du mÃłme auteur).
3. InsÃľrer 2 abonnÃľs.
4. InsÃľrer un emprunt avec une date de retour prÃľvue dans 15 jours.
5. Essayer d'insÃľrer un livre avec un `NumAut` inexistant. Qu'arrive-t-il ?
6. Essayer d'insÃľrer un livre avec `Genre = 'Poesie'`. Qu'arrive-t-il ?

3.2 UPDATE

Syntaxe UPDATE

```
UPDATE nom_table
SET col1 = val1 [, col2 = val2, ...]
[WHERE condition];
-- ATTENTION : sans WHERE, toutes les lignes sont modifiÃľes !
```

Listing 3 – Exemples UPDATE

```
-- Augmenter le salaire d'un employÃľ prÃľcis
UPDATE Employe SET Salaire = Salaire * 1.10 WHERE NSS = 1001;

-- Mettre Ãă jour plusieurs colonnes
UPDATE Employe
SET Salaire = 32000, Categorie = 'Mecanicien'
```

```sql
WHERE NSS = 1002;

-- Mise Ãă jour avec sous-requÃłte
UPDATE Employe
SET Salaire = Salaire + 5000
WHERE NSS IN (
  SELECT NumEmploye FROM Intervenants
  GROUP BY NumEmploye HAVING COUNT(*) > 5
);

-- Mettre Ãă jour Ãă partir d'une autre table (correlated update)
UPDATE Livre L
SET Prix = Prix * 1.05
WHERE EXISTS (SELECT 1 FROM Auteur A
              WHERE A.NumAut = L.NumAut AND A.Nationalite = 'FranÃğaise');
```

Exercice 4 – UPDATE

1. Augmenter de 10% le prix de tous les livres publiÃľs avant 2000.
2. Suspendre (Statut = 'Suspendu') tous les abonnÃľs qui ont un emprunt non rendu depuis plus de 30 jours (DateRetourEff IS NULL et DateRetour < SYSDATE - 30).
3. Mettre Ãă NULL la date de retour effective de tous les emprunts de l'abonnÃľ numÃľro 3.

3.3 DELETE

Syntaxe DELETE

```sql
DELETE FROM nom_table [WHERE condition];
-- Sans WHERE : supprime toutes les lignes (mais peut Ãłtre annulÃľ
   avec ROLLBACK)
```

Listing 4 – Exemples DELETE

```sql
-- Supprimer un enregistrement prÃľcis
DELETE FROM Auteur WHERE NumAut = 5;

-- Supprimer avec sous-requÃłte
DELETE FROM Emprunt
WHERE ISBN IN (SELECT ISBN FROM Livre WHERE Genre = 'Roman');

-- Supprimer les emprunts anciens (plus de 2 ans)
DELETE FROM Emprunt
WHERE DateRetourEff < SYSDATE - 730;
```

Attention

Si une FK avec **ON DELETE NO ACTION** (par dÃľfaut) existe, une tentative de suppression d'un parent rÃľfÃľrencÃľ ÃľchouerA avec ORA-02292. Il faut d'abord supprimer les enfants, ou utiliser **ON DELETE CASCADE**.

Exercice 5 – DELETE

1. Supprimer tous les livres dont le prix est NULL.
2. Supprimer l'auteur numÃľro 2 sachant que des livres lui sont associÃľs. Proposer 2 solutions diffÃľrentes.
3. Supprimer tous les emprunts dont la date de retour effective est renseignÃľe (livre rendu).

4 DQL – SELECT : Interrogation complÃĺte

4.1 Syntaxe gÃľnÃľrale

Syntaxe SELECT complet

```sql
SELECT [DISTINCT] col1 [AS alias1], col2, fonct(col3), ...
FROM   table1 [alias1], table2 [alias2]        -- jointure implicite
  -- OU jointure explicite :
  -- FROM table1 JOIN table2 ON condition
WHERE  condition_jointure AND condition_selection
GROUP BY col1, col2
HAVING condition_sur_groupe
ORDER BY col1 [ASC|DESC], col2 [ASC|DESC];
```

Ordre d'ÃľvaluaTion (important !) : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY

4.2 Fonctions d'agrÃľgation

Fonction	Description	Exemple
`COUNT(*)`	Nb de lignes	`COUNT(*) AS total`
`COUNT(col)`	Nb de valeurs non-NULL	`COUNT(Salaire)`
`SUM(col)`	Somme	`SUM(Salaire)`
`AVG(col)`	Moyenne	`AVG(Note)`
`MAX(col)`	Maximum	`MAX(Prix)`
`MIN(col)`	Minimum	`MIN(Annee)`

Listing 5 – Exemples SELECT

```sql
-- SELECT simple avec alias
SELECT NomAut || ' ' || PrenomAut AS NomComplet, Nationalite
FROM Auteur ORDER BY NomAut;

-- GROUP BY avec HAVING
SELECT Genre, COUNT(*) AS NbLivres, AVG(Prix) AS PrixMoyen
FROM Livre
GROUP BY Genre
HAVING COUNT(*) > 2
```

```sql
ORDER BY NbLivres DESC;

-- Jointure implicite (style classique)
SELECT L.Titre, A.NomAut, L.Prix
FROM Livre L, Auteur A
WHERE L.NumAut = A.NumAut AND L.Genre = 'Informatique';

-- Jointure explicite (style moderne)
SELECT L.Titre, A.NomAut
FROM Livre L JOIN Auteur A ON L.NumAut = A.NumAut;

-- LEFT JOIN (inclut les livres sans auteur)
SELECT L.Titre, A.NomAut
FROM Livre L LEFT JOIN Auteur A ON L.NumAut = A.NumAut;

-- Sous-requÃłte dans WHERE
SELECT NomAut FROM Auteur
WHERE NumAut IN (SELECT NumAut FROM Livre WHERE Prix > 1500);

-- Sous-requÃłte corrÃľlÃľe (EXISTS)
SELECT NomAbo FROM Abonne A
WHERE EXISTS (SELECT 1 FROM Emprunt E WHERE E.NumAbo = A.NumAbo
              AND E.DateRetourEff IS NULL);

-- MAX dans sous-requÃłte (employÃľ avec le plus grand salaire)
SELECT Nom, Salaire FROM Employe
WHERE Salaire = (SELECT MAX(Salaire) FROM Employe);

-- Sous-requÃłte avec MAX COUNT (employee avec le plus d'interventions)
SELECT NumEmploye, COUNT(*) AS NbInterv
FROM Intervenants
GROUP BY NumEmploye
HAVING COUNT(*) = (SELECT MAX(COUNT(*)) FROM Intervenants GROUP BY
    NumEmploye);
```

4.3 Fonctions SQL utiles

Listing 6 – Fonctions de manipulation

```sql
-- Dates
SELECT SYSDATE,
       TO_DATE('15/06/2024','DD/MM/YYYY') AS d1,
       SYSDATE + 30 AS dans30j,
       SYSDATE - TO_DATE('01/01/2024','DD/MM/YYYY') AS nbJours,
       ADD_MONTHS(SYSDATE, 6) AS dans6mois,
       EXTRACT(YEAR FROM SYSDATE) AS annee,
       TO_CHAR(SYSDATE, 'DD/MM/YYYY') AS dateChaine
FROM DUAL;

-- ChaÃőnes
SELECT UPPER('hello'),          -- HELLO
       LOWER('WORLD'),          -- world
       TRIM('  oracle  '),      -- oracle
       SUBSTR('Oracle',1,3),    -- Ora
       LENGTH('Oracle'),        -- 6
       INSTR('Oracle','a'),     -- 3
```

```
       REPLACE('Java','a','e')  -- Jeve
FROM DUAL;

-- Nombres
SELECT ROUND(3.14159, 2),   -- 3.14
       TRUNC(3.9),          -- 3
       MOD(10, 3),          -- 1
       ABS(-5)              -- 5
FROM DUAL;

-- NULL
SELECT NVL(Salaire, 0) FROM Employe;           -- remplace NULL par 0
SELECT NVL2(Salaire, 'Connu', 'Inconnu') FROM Employe;  -- if/else sur
    NULL
SELECT COALESCE(Tel1, Tel2, 'N/A') FROM Client;          -- 1er non-NULL
```

Exercice 6 – SELECT

Avec la BD bibliothÃĺque :

1. Lister tous les livres avec leur auteur (Titre, NomAut, Nationalite).
2. Donner le nombre de livres par genre, triÃľs par nombre dÃľcroissant.
3. Trouver le livre le plus cher par genre.
4. Lister les auteurs qui ont Ãľcrit plus de 2 livres.
5. Lister les abonnÃľs qui n'ont jamais empruntÃľ de livre.
6. Donner la liste des emprunts non rendus (DateRetourEff IS NULL) avec le nom de l'abonnÃľ et le titre du livre.
7. Calculer pour chaque abonnÃľ le nombre total d'emprunts et la moyenne des jours d'emprunt (DateRetourEff - DateEmprunt).

5 Vues et SÃľquences

5.1 Vues

Syntaxe CREATE VIEW

```
CREATE [OR REPLACE] [FORCE] VIEW nom_vue [(col1, col2, ...)]
AS
  SELECT ...
[WITH CHECK OPTION]     -- interdit les INSERT/UPDATE qui sortiraient
    de la vue
[WITH READ ONLY];       -- vue en lecture seule

-- Supprimer une vue
DROP VIEW nom_vue;
```

Listing 7 – Exemples de vues

```
-- Vue simple
CREATE OR REPLACE VIEW Vue_Livres_Info AS
```

```sql
SELECT L.ISBN, L.Titre, A.NomAut, L.Prix
FROM Livre L JOIN Auteur A ON L.NumAut = A.NumAut
WHERE L.Genre = 'Informatique';

-- Utiliser la vue comme une table
SELECT * FROM Vue_Livres_Info WHERE Prix < 2000;

-- Vue en lecture seule
CREATE VIEW Vue_Stats_Emprunts AS
SELECT NumAbo, COUNT(*) AS NbEmprunts
FROM Emprunt GROUP BY NumAbo
WITH READ ONLY;
```

5.2 SÃľquences

Syntaxe CREATE SEQUENCE

```sql
CREATE SEQUENCE nom_seq
  [START WITH valeur_debut]      -- dÃľfaut : 1
  [INCREMENT BY pas]             -- dÃľfaut : 1
  [MINVALUE val | NOMINVALUE]
  [MAXVALUE val | NOMAXVALUE]
  [CYCLE | NOCYCLE]              -- recommencer ou pas
  [CACHE n | NOCACHE];           -- prÃľ-gÃľnÃľrer n valeurs en
     mÃľmoire

-- Utilisation
SELECT nom_seq.NEXTVAL FROM DUAL;    -- prochain numÃľro
SELECT nom_seq.CURRVAL FROM DUAL;    -- numÃľro actuel (aprÃís un
    NEXTVAL)

INSERT INTO Auteur VALUES (seq_auteur.NEXTVAL, 'Camus', 'Albert', '
    FranÃğaise');
```

Exercice 7 – Vues et SÃľquences

1. CrÃľer une sÃľquence `seq_livre` commenÃğant Ãă 100, avec un pas de 10.
2. CrÃľer une vue `V_Emprunts_EnCours` affichant les emprunts non rendus avec le nom de l'abonnÃľ, le titre du livre et la date de retour prÃľvue.
3. InsÃľrer 2 livres en utilisant la sÃľquence pour l'ISBN.
4. Essayer de faire un UPDATE sur la vue. Que se passe-t-il ?

6 Index

Un index accÃľlÃíre les recherches sur une colonne au prix d'un espace disque supplÃľmentaire et d'un coÃżt de maintenance lors des insertions/mises Ãă jour.

6.1 Index B-arbre (par dÃľfaut)

Syntaxe CREATE INDEX B-arbre

```
-- Index simple
CREATE INDEX nom_idx ON nom_table (col);

-- Index unique (Ãľquivalent Ãă une contrainte UNIQUE)
CREATE UNIQUE INDEX nom_idx ON nom_table (col);

-- Index composÃľ (sur plusieurs colonnes)
CREATE INDEX nom_idx ON nom_table (col1, col2);

-- Index sur expression
CREATE INDEX nom_idx ON nom_table (UPPER(Nom));

-- Supprimer un index
DROP INDEX nom_idx;
```

Quand utiliser un index B-arbre ? Sur les colonnes Ãă **haute cardinalitÃľ** (beaucoup de valeurs distinctes) : NSS, email, dates, noms...

6.2 Index Bitmap

Syntaxe CREATE BITMAP INDEX

```
CREATE BITMAP INDEX nom_idx ON nom_table (col);
```

Quand utiliser un index Bitmap ? Sur les colonnes Ãă **faible cardinalitÃľ** (peu de valeurs distinctes) : Genre, Statut, Sexe, Pays... TrÃĺs efficace pour les requÃłtes avec plusieurs conditions AND/OR sur des attributs de ce type.

Taille d'un index Bitmap :

$$\text{Taille (bits)} = ||T|| \times \text{NDIST(col)}$$

$$\text{Taille (Ko)} = \frac{||T|| \times \text{NDIST(col)}}{8 \times 1024}$$

Exemple : Table LIVRE avec 10 000 tuples, attribut Genre avec 4 valeurs distinctes.

$$\text{Taille} = \frac{10000 \times 4}{8 \times 1024} \approx 4.88 \text{ Ko}$$

6.3 Index sur clÃľ ÃľtrangÃĺre (bonne pratique)

Listing 8 – Exemples d'index

```
-- Index sur FK pour accÃľlÃľrer les jointures
CREATE INDEX idx_livre_aut ON Livre(NumAut);
CREATE INDEX idx_emprunt_abo ON Emprunt(NumAbo);
```

```sql
CREATE INDEX idx_emprunt_isbn ON Emprunt(ISBN);

-- Index bitmap sur Genre (4 valeurs distinctes)
CREATE BITMAP INDEX idx_bmp_genre ON Livre(Genre);

-- Index bitmap sur Statut (2 valeurs : Actif / Suspendu)
CREATE BITMAP INDEX idx_bmp_statut ON Abonne(Statut);

-- Voir les index existants
SELECT INDEX_NAME, TABLE_NAME, INDEX_TYPE, UNIQUENESS
FROM USER_INDEXES;
```

Exercice 8 – Index

1. CrÃľer un index B-arbre sur `Livre.Titre`.
2. CrÃľer un index bitmap sur `Abonne.Statut`.
3. Calculer la taille en Ko de l'index bitmap sur `Emprunt` si la table contient 50 000 tuples et que `Statut` a 3 valeurs distinctes.
4. Pourquoi ne faut-il PAS crÃľer un bitmap sur `Livre.ISBN` ?
5. Afficher tous les index de l'utilisateur courant.

7 Tablespaces

Syntaxe Tablespace permanent

```sql
CREATE [BIGFILE | SMALLFILE] TABLESPACE nom_ts
  DATAFILE 'chemin/fichier.dbf' SIZE n [K|M|G]
  [AUTOEXTEND {OFF | ON [NEXT n [K|M|G]] [MAXSIZE {UNLIMITED | n}]}]
  [EXTENT MANAGEMENT LOCAL {AUTOALLOCATE | UNIFORM [SIZE n]}]
  [SEGMENT SPACE MANAGEMENT {AUTO | MANUAL}]
  [LOGGING | NOLOGGING]
  [ONLINE | OFFLINE];
```

Syntaxe Tablespace temporaire

```sql
CREATE TEMPORARY TABLESPACE nom_temp
  TEMPFILE 'chemin/temp.dbf' SIZE n [K|M|G]
  [AUTOEXTEND ON [NEXT n] [MAXSIZE UNLIMITED]];
```

Listing 9 – OpÃľrations sur tablespace

```sql
-- Mettre OFFLINE / ONLINE
ALTER TABLESPACE mon_ts OFFLINE;
ALTER TABLESPACE mon_ts ONLINE;

-- Ajouter un fichier
ALTER TABLESPACE mon_ts ADD DATAFILE 'fichier2.dbf' SIZE 50M;

-- ÃĽtendre un fichier
ALTER DATABASE DATAFILE 'fichier.dbf' RESIZE 200M;
```

```
-- Passer en lecture seule
ALTER TABLESPACE mon_ts READ ONLY;

-- Supprimer (avec son contenu et ses fichiers)
DROP TABLESPACE mon_ts INCLUDING CONTENTS AND DATAFILES CASCADE
    CONSTRAINTS;

-- Consulter les tablespaces
SELECT TABLESPACE_NAME, STATUS, CONTENTS FROM DBA_TABLESPACES;

-- Attribuer un tablespace ÃĂ une table
CREATE TABLE MaTable (...) TABLESPACE mon_ts;
```

Exercice 9 – Tablespaces

1. CrÃľer un tablespace permanent `BIBLIO_TBS` de 200 Mo, auto-extensible avec un incrÃľment de 50 Mo.
2. CrÃľer un tablespace temporaire `BIBLIO_TEMP` de 50 Mo.
3. CrÃľer un utilisateur `BIBLIOTHECAIRE` qui utilise ces tablespaces, avec un quota de 100 Mo sur `BIBLIO_TBS`.
4. Ajouter un fichier de donnÃľes de 100 Mo au tablespace `BIBLIO_TBS`.
5. Afficher tous les tablespaces de la base.

8 Utilisateurs et PrivilÃĺges

8.1 Gestion des utilisateurs

Syntaxe CREATE USER

```
CREATE USER nom_user
  IDENTIFIED BY mot_de_passe
  [DEFAULT TABLESPACE nom_ts]
  [QUOTA {n [K|M] | UNLIMITED} ON nom_ts]
  [TEMPORARY TABLESPACE nom_temp]
  [PASSWORD EXPIRE]            -- oblige ÃĂ changer le mdp ÃĂ la 1
      ÃĺÃĺre connexion
  [ACCOUNT {LOCK | UNLOCK}]    -- verrouiller le compte
  [PROFILE nom_profil];
```

Listing 10 – CrÃľer, modifier, supprimer un utilisateur

```
-- CrÃľation complÃĺte
CREATE USER etudiant01 IDENTIFIED BY pass2024
  DEFAULT TABLESPACE USERS  QUOTA 50M ON USERS
  TEMPORARY TABLESPACE TEMP
  PASSWORD EXPIRE
  ACCOUNT UNLOCK;

-- Modifier le mot de passe
```

```sql
ALTER USER etudiant01 IDENTIFIED BY nouveaupass;

-- Verrouiller / dÃľverrouiller
ALTER USER etudiant01 ACCOUNT LOCK;
ALTER USER etudiant01 ACCOUNT UNLOCK;

-- Modifier le quota
ALTER USER etudiant01 QUOTA UNLIMITED ON USERS;

-- Forcer l'expiration du mot de passe
ALTER USER etudiant01 PASSWORD EXPIRE;

-- Supprimer (CASCADE supprime tous ses objets)
DROP USER etudiant01 CASCADE;

-- Voir les utilisateurs
SELECT USERNAME, ACCOUNT_STATUS, DEFAULT_TABLESPACE, PROFILE
FROM DBA_USERS;
```

8.2 PrivilÃĺges systÃĺme

Les privilÃĺges systÃĺme permettent d'effectuer des opÃľrations gÃľnÃľrales sur la BD.

Syntaxe GRANT / REVOKE systÃĺme

```sql
GRANT privilege_sys [, ...] TO {user | role | PUBLIC}
  [WITH ADMIN OPTION];   -- le bÃľnÃľficiaire peut re-donner ce
      privilÃĺge

REVOKE privilege_sys [, ...] FROM {user | role | PUBLIC};
```

Listing 11 – Exemples privilÃĺges systÃĺme

```sql
-- Permettre la connexion
GRANT CREATE SESSION TO etudiant01;

-- Permettre la crÃľation d'objets
GRANT CREATE TABLE, CREATE VIEW, CREATE INDEX TO etudiant01;
GRANT CREATE ANY TABLE TO etudiant01;   -- dans n'importe quel schÃľma

-- Donner tous les privilÃĺges (= admin)
GRANT ALL PRIVILEGES TO etudiant01;

-- Avec ADMIN OPTION (peut re-donner)
GRANT CREATE TABLE TO etudiant01 WITH ADMIN OPTION;

-- RÃľvoquer
REVOKE CREATE TABLE FROM etudiant01;
```

8.3 PrivilÃĺges objet

Les privilÃĺges objet contrÃťlent l'accÃĺs Ãă un objet prÃľcis (table, vue, sÃľquence...).

Syntaxe GRANT / REVOKE objet

```
GRANT {priv1 [, priv2] | ALL PRIVILEGES} ON [schema.]objet
  TO {user | role | PUBLIC}
  [WITH GRANT OPTION];    -- le bÃľnÃľficiaire peut re-donner ce
      privilÃíge

REVOKE {priv1 | ALL PRIVILEGES} ON [schema.]objet
  FROM {user | role | PUBLIC};
```

Listing 12 – Exemples privilÃíges objet

```
-- Lecture seule
GRANT SELECT ON DBAINTERVENTION.Employe TO etudiant01;

-- Lecture + mise Ãă jour de colonnes spÃľcifiques
GRANT SELECT, UPDATE (Salaire, Categorie) ON Employe TO etudiant01;

-- Tous les droits
GRANT ALL PRIVILEGES ON Commande TO gestionnaire;

-- Accessible Ãă tous
GRANT SELECT ON Filiale TO PUBLIC;

-- Avec GRANT OPTION
GRANT SELECT ON Livre TO etudiant01 WITH GRANT OPTION;

-- RÃľvoquer
REVOKE SELECT ON Employe FROM etudiant01;
REVOKE ALL PRIVILEGES ON Commande FROM gestionnaire;
```

PrivilÃíge	Table	Vue	SÃľquence	ProcÃľdure	Index
SELECT	✓	✓	✓		
INSERT	✓	✓			
UPDATE	✓	✓			
DELETE	✓	✓			
ALTER	✓		✓		
INDEX	✓				✓
REFERENCES	✓	✓	✓		
EXECUTE				✓	

Exercice 10 – Utilisateurs et PrivilÃíges

1. CrÃľer un utilisateur `LECTEUR` qui peut se connecter et lire toutes les tables de la BD bibliothÃíque (pas modifier).
2. CrÃľer un utilisateur `GESTIONNAIRE` qui peut tout faire sur les tables `LIVRE` et `AUTEUR`, et uniquement lire `EMPRUNT`.
3. RÃľvoquer le droit SELECT sur LIVRE Ãă `LECTEUR`.
4. VÃľrifier les privilÃíges accordÃľs Ãă `LECTEUR` en interrogeant le dictionnaire.

9 RÃťles

Un rÃťle est un groupe nommÃľ de privilÃĺges. On donne le rÃťle aux utilisateurs au lieu de leur donner les privilÃĺges un par un.

Syntaxe RÃťles

```sql
-- CrÃľer un rÃťle
CREATE ROLE nom_role [NOT IDENTIFIED | IDENTIFIED BY mdp];

-- Donner des privilÃĺges au rÃťle
GRANT SELECT ON Livre TO nom_role;
GRANT CREATE TABLE   TO nom_role;

-- Donner le rÃťle Ãă un utilisateur
GRANT nom_role TO nom_user;

-- Activer/dÃľsactiver un rÃťle dans la session
SET ROLE nom_role;
SET ROLE ALL EXCEPT nom_role;
SET ROLE NONE;

-- Supprimer un rÃťle
DROP ROLE nom_role;

-- Voir les rÃťles
SELECT ROLE FROM DBA_ROLES;
SELECT GRANTEE, GRANTED_ROLE FROM DBA_ROLE_PRIVS WHERE GRANTEE = '
    MON_USER';
```

RÃťles Oracle prÃľdÃľfinis :

RÃťle	Description
`CONNECT`	Connexion basique (CREATE SESSION + quelques autres)
`RESOURCE`	CrÃľer des objets (tables, index, sÃľquences, procÃľdures...)
`DBA`	Tous les privilÃĺges systÃĺme avec ADMIN OPTION

Exercice 11 – RÃťles

1. CrÃľer un rÃťle `ROLE_LECTURE` avec SELECT sur toutes les tables de la BD bibliothÃĺque.
2. CrÃľer un rÃťle `ROLE_GESTION` avec INSERT, UPDATE, DELETE sur `LIVRE` et `AUTEUR`.
3. Attribuer `ROLE_LECTURE` Ãă l'utilisateur `LECTEUR` et `ROLE_GESTION` Ãă `GESTIONNAIRE`.
4. Retirer `ROLE_GESTION` Ãă `GESTIONNAIRE`.

10 Profils

Un profil impose des limites sur l'utilisation des ressources et la gestion des mots de passe.

Syntaxe CREATE PROFILE

```
CREATE PROFILE nom_profil LIMIT
  -- âŤĂâŤĂ Ressources systÃłme
     âŤĂâŤĂâŤĂâ
```

```
  PASSWORD_LIFE_TIME     30
  PASSWORD_REUSE_TIME    180;

-- Modifier un profil
ALTER PROFILE PROFIL_ETUDIANT LIMIT CONNECT_TIME 180;

-- Affecter un profil Ãă un utilisateur
ALTER USER etudiant01 PROFILE PROFIL_ETUDIANT;

-- Supprimer (impossible pour DEFAULT)
DROP PROFILE PROFIL_ETUDIANT;    -- seuls les profils non assignÃľs

-- Voir les profils
SELECT PROFILE, RESOURCE_NAME, LIMIT FROM DBA_PROFILES
WHERE PROFILE = 'PROFIL_ETUDIANT' ORDER BY RESOURCE_NAME;
```

Exercice 12 – Profils

1. CrÃľer un profil `PROFIL_BIBLIO` avec les contraintes suivantes :
   — Max 2 sessions simultanÃľes
   — DÃľconnexion aprÃĺs 45 min d'inactivitÃľ
   — Blocage aprÃĺs 4 tentatives incorrectes pendant 2 jours
   — Mot de passe expire aprÃĺs 60 jours avec 5 jours de grÃćce
2. Appliquer ce profil Ãă l'utilisateur `BIBLIOTHECAIRE`.
3. Modifier le profil pour augmenter la durÃľe d'inactivitÃľ Ãă 60 min.
4. Afficher les paramÃĺtres du profil `PROFIL_BIBLIO`.

11 PL/SQL – Introduction et Blocs

PL/SQL (*Procedural Language / SQL*) est l'extension procÃľdurale de SQL d'Oracle. Il permet d'ÃľcrIre des programmes contenant des boucles, des conditions, des exceptions, etc.

11.1 Structure d'un bloc PL/SQL

Syntaxe Bloc PL/SQL

```
DECLARE
  -- DÃľclaration des variables, curseurs, exceptions
  variable1  type1 [:= valeur_initiale];
  variable2  type2 DEFAULT valeur;
BEGIN
  -- Instructions exÃľcutables (SQL + contrÃľle de flux)
  instruction1;
  instruction2;
EXCEPTION
  WHEN exception1 THEN
    -- Traitement de l'erreur
  WHEN OTHERS THEN
```

```
    -- Toute autre erreur
END;
/   -- Le slash exÃľcute le bloc dans SQL*Plus
```

Activation de l'affichage :

```
SET SERVEROUTPUT ON    -- Ãă exÃľcuter avant tout bloc PL/SQL
```

11.2 DÃľclaration de variables

Listing 14 – Types de variables PL/SQL

```
DECLARE
  -- Types scalaires SQL
  v_nom         VARCHAR2(50);
  v_salaire     NUMBER(10,2);
  v_date        DATE;
  v_actif       BOOLEAN := TRUE;

  -- Initialisation
  v_compteur    NUMBER := 0;
  v_msg         VARCHAR2(100) DEFAULT 'Bonjour';
  v_pi          CONSTANT NUMBER := 3.14159;    -- constante

  -- RÃľfÃľrence au type d'une colonne (%TYPE)
  v_sal_emp     Employe.Salaire%TYPE;           -- mÃłme type que Employe.
     Salaire
  v_nom_emp     Employe.Nom%TYPE;

  -- RÃľfÃľrence au type d'une ligne (%ROWTYPE)
  v_emp         Employe%ROWTYPE;                -- toute la structure de la
     table

BEGIN
  v_nom := 'Ahmed';
  v_salaire := 35000.50;
  v_emp.Nom := 'Benali';                        -- accÃís aux champs

  DBMS_OUTPUT.PUT_LINE('Nom : ' || v_nom);
  DBMS_OUTPUT.PUT_LINE('Salaire : ' || TO_CHAR(v_salaire));
END;
/
```

Note importante

%TYPE est trÃís utile : si le type de la colonne change dans la table, la variable s'adapte automatiquement. Toujours prÃľfÃľrer `Employe.Salaire%TYPE` Ãă `NUMBER(10,2)`.

11.3 SELECT INTO (mono-ligne)

Listing 15 – SELECT INTO

```
DECLARE
```

```
  v_nom     Employe.Nom%TYPE;
  v_sal     Employe.Salaire%TYPE;
  v_emp     Employe%ROWTYPE;
BEGIN
  -- SÃľlectionner une seule ligne dans des variables
  SELECT Nom, Salaire INTO v_nom, v_sal
  FROM Employe WHERE NSS = 1001;
  DBMS_OUTPUT.PUT_LINE(v_nom || ' gagne ' || v_sal);

  -- SÃľlectionner toute une ligne
  SELECT * INTO v_emp FROM Employe WHERE NSS = 1001;
  DBMS_OUTPUT.PUT_LINE(v_emp.Nom || ' - ' || v_emp.Categorie);
END;
/
-- Si la requÃŧte ramÃĻne 0 ligne  âĘŠ exception NO_DATA_FOUND
-- Si la requÃŧte ramÃĻne > 1 ligne âĘŠ exception TOO_MANY_ROWS
```

Exercice 13 – Blocs PL/SQL de base

1. ÃĽcrire un bloc qui affiche le nom et le salaire de l'employÃľ dont le NSS est 53.
2. ÃĽcrire un bloc qui calcule et affiche la moyenne des salaires de la table EMPLOYE.
3. ÃĽcrire un bloc qui affiche « Riche » si le salaire de l'employÃľ 53 est > 20000, sinon « Modeste ».

12 Structures de ContrÃŧle PL/SQL

12.1 IF / ELSIF / ELSE

```
-- Structure simple
IF condition THEN
  instructions;
END IF;

-- Avec alternative
IF condition THEN
  instructions;
ELSE
  autres_instructions;
END IF;

-- Avec plusieurs branches
IF note >= 16 THEN
  mention := 'TrÃĻs Bien';
ELSIF note >= 14 THEN
  mention := 'Bien';
ELSIF note >= 12 THEN
  mention := 'Assez Bien';
ELSIF note >= 10 THEN
  mention := 'Passable';
ELSE
  mention := 'AjournÃľ';
END IF;
```

12.2 CASE

```
-- CASE sur une valeur (switch)
CASE v_grade
  WHEN 'TS'                THEN v_coeff := 1.0;
  WHEN 'Ingenieur'         THEN v_coeff := 1.5;
  WHEN 'Ingenieur Principal' THEN v_coeff := 2.0;
  WHEN 'Administrateur'    THEN v_coeff := 1.8;
  ELSE                          v_coeff := 1.2;
END CASE;

-- CASE recherchÃľ (conditions libres)
CASE
  WHEN v_salaire < 15000 THEN DBMS_OUTPUT.PUT_LINE('Bas');
  WHEN v_salaire < 30000 THEN DBMS_OUTPUT.PUT_LINE('Moyen');
  ELSE                        DBMS_OUTPUT.PUT_LINE('Haut');
END CASE;
```

12.3 Boucles

Listing 16 – Les 3 types de boucles

```
-- 1. Boucle FOR (nombre de tours connu)
FOR i IN 1..10 LOOP
  DBMS_OUTPUT.PUT_LINE('Tour : ' || i);
END LOOP;

-- FOR en dÃľcroissant
FOR i IN REVERSE 1..10 LOOP
  DBMS_OUTPUT.PUT_LINE(i);
END LOOP;

-- 2. Boucle WHILE (condition en entrÃľe)
v_compteur := 1;
WHILE v_compteur <= 5 LOOP
  DBMS_OUTPUT.PUT_LINE('Compteur : ' || v_compteur);
  v_compteur := v_compteur + 1;
END LOOP;

-- 3. Boucle LOOP simple (EXIT WHEN = sortie conditionnelle)
LOOP
  v_val := v_val * 2;
  EXIT WHEN v_val > 1000;    -- sort si vrai
END LOOP;
```

Exercice 14 – Structures de contrÃľle

1. ÃĽcrire un bloc PL/SQL qui affiche les 10 premiers multiples de 7 (en utilisant une boucle FOR).
2. ÃĽcrire un bloc qui affiche pour chaque employÃľ de la BD : son nom et sa catÃľgorie de salaire (« Bas » si < 15000, « Moyen » si entre 15000 et 25000, « Haut » sinon). Utiliser un curseur FOR (voir section suivante).
3. ÃĽcrire un bloc qui calcule la factorielle de 8 avec une boucle WHILE.

4. ÃĽcrire un bloc qui double un nombre jusqu'Ăă ce qu'il dÃĽpasse 10000, en affichant chaque ÃĽtape.

13 Curseurs PL/SQL

Un curseur est nÃĽcessaire quand une requÃłte SELECT ramÃĺne **plusieurs lignes**. Il gÃĺre un pointeur vers la ligne courante dans le rÃĽsultat.

13.1 Curseur explicite

Syntaxe Curseur explicite

```
DECLARE
  CURSOR nom_curseur IS requete_SELECT;
  v_var1  type1;
  v_var2  type2;
BEGIN
  OPEN nom_curseur;            -- exÃĽcute la requÃłte, charge en
      mÃĽmoire SGA
  FETCH nom_curseur INTO v_var1, v_var2;  -- ramÃĺne la 1Ãĺre ligne
  WHILE nom_curseur%FOUND LOOP
    -- traitement
    FETCH nom_curseur INTO v_var1, v_var2;  -- ligne suivante
  END LOOP;
  CLOSE nom_curseur;           -- libÃĺre la mÃĽmoire SGA
END;
```

Attributs des curseurs :

Attribut	Type	Signification
cur%FOUND	BOOLEAN	Vrai si le dernier FETCH a ramenÃĽ une ligne
cur%NOTFOUND	BOOLEAN	OpposÃĽ de %FOUND
cur%ROWCOUNT	NUMBER	Nombre de lignes dÃĽjÃă ramenÃĽes
cur%ISOPEN	BOOLEAN	Vrai si le curseur est ouvert

Listing 17 – Exemple curseur complet

```
SET SERVEROUTPUT ON;
DECLARE
  CURSOR c_emp IS
    SELECT NSS, Nom, Salaire FROM Employe
    WHERE Categorie = 'Mecanicien'
    ORDER BY Salaire DESC;

  v_nss     Employe.NSS%TYPE;
  v_nom     Employe.Nom%TYPE;
  v_sal     Employe.Salaire%TYPE;
  v_total   NUMBER := 0;
  v_nb      NUMBER := 0;
```

```
BEGIN
  OPEN c_emp;
  FETCH c_emp INTO v_nss, v_nom, v_sal;

  WHILE c_emp%FOUND LOOP
    v_total := v_total + v_sal;
    v_nb    := v_nb + 1;
    DBMS_OUTPUT.PUT_LINE(v_nom || ' : ' || v_sal || ' DA');
    FETCH c_emp INTO v_nss, v_nom, v_sal;
  END LOOP;

  CLOSE c_emp;
  DBMS_OUTPUT.PUT_LINE('--- Total : ' || v_total);
  DBMS_OUTPUT.PUT_LINE('--- Nb : ' || v_nb);
  IF v_nb > 0 THEN
    DBMS_OUTPUT.PUT_LINE('--- Moy : ' || ROUND(v_total/v_nb, 2));
  END IF;
END;
/
```

13.2 Curseur avec paramÃĺtre

```
DECLARE
  CURSOR c_cat (p_cat VARCHAR2) IS
    SELECT Nom, Salaire FROM Employe WHERE Categorie = p_cat;

  v_nom Employe.Nom%TYPE;
  v_sal Employe.Salaire%TYPE;
BEGIN
  OPEN c_cat('Assistant');
  FETCH c_cat INTO v_nom, v_sal;
  WHILE c_cat%FOUND LOOP
    DBMS_OUTPUT.PUT_LINE(v_nom || ' : ' || v_sal);
    FETCH c_cat INTO v_nom, v_sal;
  END LOOP;
  CLOSE c_cat;
END;
/
```

13.3 Boucle FOR sur curseur (syntaxe simplifiÃľe)

La boucle FOR sur curseur ouvre, parcourt et ferme automatiquement le curseur.

Listing 18 – Curseur FOR LOOP – la faÃğon la plus simple

```
BEGIN
  FOR rec IN (SELECT Nom, Salaire FROM Employe ORDER BY Salaire DESC) LOOP
    DBMS_OUTPUT.PUT_LINE(rec.Nom || ' : ' || rec.Salaire);
  END LOOP;
END;
/

-- Avec curseur nommÃľ
DECLARE
  CURSOR c IS SELECT * FROM Employe WHERE Salaire > 20000;
```

```
BEGIN
  FOR emp IN c LOOP
    DBMS_OUTPUT.PUT_LINE(emp.Nom || ' (' || emp.Categorie || ')');
  END LOOP;
END;
/
```

Exercice 15 – Curseurs

1. ÃĽcrire un bloc avec un curseur explicite (OPEN/FETCH/CLOSE) qui affiche pour chaque livre son titre, son auteur et son prix.
2. ÃĽcrire un bloc avec un curseur paramÃľtrÃľ qui prend une nationalitÃľ en paramÃÍtre et affiche tous les auteurs de cette nationalitÃľ avec le nombre de livres qu'ils ont ÃĽcrits.
3. ÃĽcrire un bloc avec une boucle FOR sur curseur qui insÃÍre dans une table `LIVRE_CHER` (Titre, Prix) tous les livres dont le prix est supÃľrieur Ãă la moyenne.
4. ÃĽcrire un bloc qui parcourt les abonnÃľs et affiche pour chacun : son nom et « Actif (n emprunts) » ou « Jamais empruntÃľ ».

14 ProcÃľdures et Fonctions StockÃľes

14.1 ProcÃľdures

Syntaxe CREATE PROCEDURE

```
CREATE [OR REPLACE] PROCEDURE nom_proc
  [(param1 [IN|OUT|IN OUT] type1,
    param2 [IN|OUT|IN OUT] type2)]
AS
  -- DÃľclarations
  variable locale type;
BEGIN
  -- Corps
EXCEPTION
  -- Gestion des erreurs
END [nom_proc];
/
```

Modes de passage :

- `IN` (dÃľfaut) : valeur en entrÃľe seulement, non modifiable.
- `OUT` : paramÃÍtre de sortie, la procÃľdure y ÃĽcrit le rÃľsultat.
- `IN OUT` : en entrÃľe et en sortie.

Listing 19 – Exemples de procÃľdures

```
-- ProcÃľdure sans paramÃÍtre OUT
CREATE OR REPLACE PROCEDURE afficher_employes AS
BEGIN
  FOR emp IN (SELECT Nom, Salaire FROM Employe) LOOP
```

```
    DBMS_OUTPUT.PUT_LINE(emp.Nom || ' : ' || emp.Salaire);
  END LOOP;
END;
/

-- ExÃ©cuter la procÃ©dure
EXECUTE afficher_employes;
-- ou depuis PL/SQL :
BEGIN afficher_employes; END; /

-- ProcÃ©dure avec paramÃ¨tre OUT
CREATE OR REPLACE PROCEDURE get_salaire
  (p_nss IN Employe.NSS%TYPE, p_sal OUT Employe.Salaire%TYPE)
AS
BEGIN
  SELECT Salaire INTO p_sal FROM Employe WHERE NSS = p_nss;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    p_sal := -1;
END;
/

-- Appel avec variable de sortie
DECLARE v_sal NUMBER; BEGIN
  get_salaire(1001, v_sal);
  DBMS_OUTPUT.PUT_LINE('Salaire = ' || v_sal);
END; /

-- ProcÃ©dure d'augmentation de salaire
CREATE OR REPLACE PROCEDURE augmenter_salaire
  (p_nss IN Employe.NSS%TYPE, p_taux IN NUMBER)
AS
  v_sal Employe.Salaire%TYPE;
BEGIN
  SELECT Salaire INTO v_sal FROM Employe WHERE NSS = p_nss;
  UPDATE Employe SET Salaire = Salaire * (1 + p_taux/100) WHERE NSS =
      p_nss;
  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Nouveau salaire : ' || ROUND(v_sal*(1+p_taux/100)
      ,2));
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Employe introuvable');
END;
/
```

14.2 Fonctions

Syntaxe CREATE FUNCTION

```
CREATE [OR REPLACE] FUNCTION nom_fonction
  [(param1 type1, param2 type2, ...)]
RETURN type_retour
IS
  -- DÃ©clarations
```

```
BEGIN
  -- Corps
  RETURN valeur;    -- au moins un RETURN obligatoire
EXCEPTION
  -- Gestion des erreurs
  RETURN valeur_defaut;
END [nom_fonction];
/
```

Listing 20 – Exemples de fonctions

```
-- Fonction qui retourne le salaire moyen d'une catÃľgorie
CREATE OR REPLACE FUNCTION moy_salaire_categorie
  (p_cat Employe.Categorie%TYPE)
RETURN NUMBER
IS
  v_moy NUMBER;
BEGIN
  SELECT AVG(Salaire) INTO v_moy
  FROM Employe WHERE Categorie = p_cat;
  RETURN NVL(v_moy, 0);
EXCEPTION
  WHEN OTHERS THEN RETURN 0;
END;
/

-- Appel dans SQL
SELECT moy_salaire_categorie('Mecanicien') FROM DUAL;
SELECT Nom FROM Employe
WHERE Salaire > moy_salaire_categorie('Mecanicien');

-- Appel dans PL/SQL
DECLARE v NUMBER;
BEGIN
  v := moy_salaire_categorie('Assistant');
  DBMS_OUTPUT.PUT_LINE('Moy assistants : ' || v);
END; /

-- Fonction qui formate un nom complet
CREATE OR REPLACE FUNCTION nom_complet
  (p_nom VARCHAR2, p_prenom VARCHAR2)
RETURN VARCHAR2
IS
BEGIN
  RETURN UPPER(p_nom) || ' ' || INITCAP(p_prenom);
END;
/
SELECT nom_complet(NomAut, PrenomAut) FROM Auteur;
```

Exercice 16 – ProcÃľdures et Fonctions

1. ÃĽcrire une procÃľdure `inscrire_abonne(p_nom, p_email)` qui insÃĺre un nouvel abonnÃľ avec la date du jour.
2. ÃĽcrire une procÃľdure `enregistrer_emprunt(p_numabo, p_isbn)` qui insÃĺre un emprunt. La date de retour prÃľvue doit Ãłtre dans 14 jours.

3. ÃĽcrire une fonction `nb_emprunts_abonne(p_numabo)` qui retourne le nombre total d'emprunts d'un abonnÃĺ.
4. ÃĽcrire une fonction `est_disponible(p_isbn)` qui retourne TRUE si le livre n'est pas actuellement empruntÃĺ (DateRetourEff IS NULL), FALSE sinon.
5. ÃĽcrire une procÃĺdure `retourner_livre(p_numabo, p_isbn)` qui met Ãă jour la date de retour effective avec SYSDATE.

15 Triggers (DÃĺclencheurs)

Un trigger est exÃĺcutÃĺ automatiquement en rÃĺponse Ãă un ÃĺvÃĺnement (INSERT, UPDATE, DELETE) sur une table.

Syntaxe CREATE TRIGGER – Syntaxe complÃĺte

```oracle
CREATE [OR REPLACE] TRIGGER nom_trigger
  {BEFORE | AFTER}
  {INSERT | UPDATE [OF col1, col2] | DELETE}
  [OR INSERT | OR UPDATE | OR DELETE]     -- plusieurs ÃĺvÃĺnements
ON nom_table
[REFERENCING OLD AS ancien NEW AS nouveau]
[FOR EACH ROW]                            -- trigger ligne (sinon : globale)
[WHEN (condition)]                        -- condition supplÃĺmentaire
DECLARE
  -- variables
BEGIN
  -- corps du trigger
  -- :NEW.col   nouvelle valeur  (INSERT, UPDATE)
  -- :OLD.col   ancienne valeur  (UPDATE, DELETE)
EXCEPTION
  -- ...
END;
/
```

Types de triggers :

	BEFORE	AFTER
Statement (sans FOR EACH ROW)	Une fois avant l'opÃĺration	Une fois aprÃĺs
Row (FOR EACH ROW)	Pour chaque ligne, avant	Pour chaque ligne, aprÃĺs

:NEW et :OLD selon l'ÃĺvÃĺnement :

ÃĽvÃĺnement	:OLD	:NEW
INSERT	NULL (pas de valeur avant)	Valeur insÃĺrÃĺe
UPDATE	Ancienne valeur	Nouvelle valeur
DELETE	Valeur supprimÃĺe	NULL (pas de valeur aprÃĺs)

Listing 21 – Trigger 1 – VÃľrification de date

```
-- Le sinistre doit Ãłtre aprÃľs le contrat
CREATE OR REPLACE TRIGGER verif_date_sinistre
BEFORE INSERT OR UPDATE ON Sinistre
FOR EACH ROW
DECLARE
  v_date_c Contrat.DATE_C%TYPE;
BEGIN
  SELECT DATE_C INTO v_date_c
  FROM Contrat WHERE NOCONTRAT = :NEW.NOCONTRAT;

  IF :NEW.DATE_S < v_date_c THEN
    RAISE_APPLICATION_ERROR(-20001,
      'La date du sinistre ('||:NEW.DATE_S||') doit etre > date contrat ('
         ||v_date_c||')');
  END IF;
END;
/
```

Listing 22 – Trigger 2 – Mise Ãă jour automatique d'un agrÃľgat

```
-- Mettre Ãă jour le montant global d'une commande Ãă chaque ajout de
   ligne
CREATE OR REPLACE TRIGGER maj_montant
AFTER INSERT ON Ligne_Commande
FOR EACH ROW
DECLARE
  v_prix Produit.Prix_Unitaire%TYPE;
BEGIN
  SELECT Prix_Unitaire INTO v_prix FROM Produit WHERE NumProd = :NEW.
     NumProd;
  UPDATE Commande
  SET MontantG = MontantG + (:NEW.NB_Unite * v_prix)
  WHERE NumCMD = :NEW.NumCMD;
END;
/
```

Listing 23 – Trigger 3 – PrÃľdicats INSERTING / UPDATING / DELETING

```
-- Trigger multi-ÃľvÃľnements sur une table
CREATE OR REPLACE TRIGGER audit_employe
AFTER INSERT OR UPDATE OR DELETE ON Employe
FOR EACH ROW
BEGIN
  IF INSERTING THEN
    DBMS_OUTPUT.PUT_LINE('INSERT : ' || :NEW.Nom);

  ELSIF UPDATING THEN
    DBMS_OUTPUT.PUT_LINE('UPDATE : ' || :OLD.Nom ||
      ' - Ancien salaire : ' || :OLD.Salaire ||
      ' - Nouveau : ' || :NEW.Salaire);

    -- VÃľrifier qu'on ne diminue pas le salaire
    IF :NEW.Salaire < :OLD.Salaire THEN
      RAISE_APPLICATION_ERROR(-20002, 'Le salaire ne peut pas diminuer !')
         ;
    END IF;

```

```
  ELSIF DELETING THEN
    DBMS_OUTPUT.PUT_LINE('DELETE : ' || :OLD.Nom);
  END IF;
END;
/
```

Listing 24 – Trigger 4 – Compteur automatique (avec UPDATE de table liÃľe)

```
-- Mettre Ãă jour NbEtud dans Section quand un Ãľtudiant est insÃľrÃľ
CREATE OR REPLACE TRIGGER maj_nb_etud
AFTER INSERT ON Etudiant
FOR EACH ROW
BEGIN
  UPDATE Section
  SET NbEtud = NbEtud + 1
  WHERE CodeSection = :NEW.CodeSection;
END;
/
```

Listing 25 – Trigger 5 – BEFORE UPDATE avec condition WHEN

```
-- Ne dÃľclencher que si le salaire change et dÃľpasse un seuil
CREATE OR REPLACE TRIGGER check_salaire_max
BEFORE UPDATE OF Salaire ON Employe
FOR EACH ROW
WHEN (NEW.Salaire > 100000)   -- condition WITHOUT le ":"
BEGIN
  RAISE_APPLICATION_ERROR(-20003,
    'Salaire trop ÃľlevÃľ : ' || :NEW.Salaire || ' DA');
END;
/
```

Listing 26 – Gestion des triggers

```
-- DÃľsactiver un trigger
ALTER TRIGGER nom_trigger DISABLE;

-- RÃľactiver
ALTER TRIGGER nom_trigger ENABLE;

-- DÃľsactiver TOUS les triggers d'une table
ALTER TABLE Employe DISABLE ALL TRIGGERS;
ALTER TABLE Employe ENABLE  ALL TRIGGERS;

-- Voir les triggers
SELECT TRIGGER_NAME, TRIGGER_TYPE, TRIGGERING_EVENT, TABLE_NAME, STATUS
FROM USER_TRIGGERS;
```

Exercice 17 – Triggers

1. ÃĽcrire un trigger qui empÃłche de supprimer un auteur s'il a encore des livres dans la BD (sans utiliser ON DELETE CASCADE).
2. ÃĽcrire un trigger qui met automatiquement la date d'inscription Ãă SYSDATE lors de l'insertion d'un abonnÃľ (si elle est NULL).
3. ÃĽcrire un trigger qui interdit qu'un emprunt soit fait par un abonnÃľ avec le statut 'Suspendu'.

4. ÃĽcrire un trigger `AFTER UPDATE` sur la table EMPRUNT qui met automatiquement le statut de l'abonnÃĽ Ãă 'Actif' lorsque tous ses emprunts sont rendus (DateRetourEff IS NOT NULL pour tous ses emprunts).
5. ÃĽcrire un trigger qui enregistre dans une table `AUDIT_LOG(Operation, NomTable, DateOp, UserOp)` chaque INSERT, UPDATE ou DELETE sur la table LIVRE.

16 Gestion des Exceptions PL/SQL

Syntaxe Exceptions

```
DECLARE
  -- Exceptions Oracle prÃĽdÃĽfinies : pas de dÃĽclaration nÃĽcessaire
  -- Exceptions utilisateur avec code Oracle :
  ma_contrainte EXCEPTION;
  PRAGMA EXCEPTION_INIT(ma_contrainte, -2291);  -- -2291 = FK violation

  -- Exceptions utilisateur pures :
  salaire_invalide EXCEPTION;
BEGIN
  -- Lever une exception utilisateur
  IF v_sal < 0 THEN
    RAISE salaire_invalide;
  END IF;

  -- Lever avec message personnalisÃĽ (code entre -20000 et -20999)
  RAISE_APPLICATION_ERROR(-20001, 'Salaire negatif interdit');

EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Aucune donnee : ' || SQLERRM);

  WHEN TOO_MANY_ROWS THEN
    DBMS_OUTPUT.PUT_LINE('Trop de lignes retournÃĽes');

  WHEN DUP_VAL_ON_INDEX THEN
    DBMS_OUTPUT.PUT_LINE('Doublon : ' || SQLERRM);

  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Division par zero');

  WHEN ma_contrainte THEN
    DBMS_OUTPUT.PUT_LINE('ClÃĽ etrangere violee');

  WHEN salaire_invalide THEN
    DBMS_OUTPUT.PUT_LINE('Salaire invalide');

  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur ' || SQLCODE || ' : ' || SQLERRM);
    ROLLBACK;
```

```
END;
/
```

Exceptions prÃľdÃľfinies importantes :

Exception	Code	Cause
NO_DATA_FOUND	ORA-01403	SELECT INTO sans rÃľsultat
TOO_MANY_ROWS	ORA-01422	SELECT INTO avec >1 ligne
DUP_VAL_ON_INDEX	ORA-00001	Violation UNIQUE/PK
ZERO_DIVIDE	ORA-01476	Division par zÃľro
VALUE_ERROR	ORA-06502	Erreur de conversion/tronc.
CURSOR_ALREADY_OPEN	ORA-06511	Curseur dÃľjÃă ouvert

Listing 27 – Exemple complet avec exceptions

```
CREATE OR REPLACE PROCEDURE chercher_employe (p_nss NUMBER)
AS
  v_emp Employe%ROWTYPE;
BEGIN
  SELECT * INTO v_emp FROM Employe WHERE NSS = p_nss;
  DBMS_OUTPUT.PUT_LINE('TrouvÃľ : ' || v_emp.Nom || ', ' || v_emp.Salaire)
      ;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Aucun employe avec NSS = ' || p_nss);
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Erreur inattendue : ' || SQLERRM);
END;
/
```

Exercice 18 – Exceptions

1. ÃĽcrire une procÃľdure `diviser(a, b, resultat OUT)` qui effectue la division a/b en gÃľrant l'exception de division par zÃľro.
2. Modifier la procÃľdure `inscrire_abonne` (exo 16) pour gÃľrer : l'exception de doublon sur l'email (si l'email est unique), et toute autre erreur Oracle.
3. ÃĽcrire un bloc qui essaie de sÃľlectionner un employÃľ avec un NSS saisi par l'utilisateur, et affiche un message clair pour NO_DATA_FOUND et TOO_MANY_ROWS.

17 Dictionnaire de DonnÃľes Oracle

Le dictionnaire de donnÃľes est une mÃľta-base qui dÃľcrit tous les objets de la BD.

Listing 28 – RequÃłtes utiles sur le dictionnaire

```
-- âŤĂâŤĂ Voir mes tables
   âŤĂâŤĂâŤĂâ
```

```sql
SELECT TABLE_NAME FROM USER_TABLES;

-- âŤĂâŤĂ Colonnes d'une table
   âŤĂâŤĂâŤĂâ
```

```sql

-- âŤĂâŤĂ Taille d'une table en Ko
   âŤĂâŤĂâŤĂâ
```